2026-05-05-p1-f09-slash-command-system-design

Phase 1 / Feature 9 — Slash Command System (user-defined Markdown commands)

Date: 2026-05-05 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

---

1. Goal

Add a parallel slash-command surface for **user-defined Markdown commands**, distinct from the built-in Go-defined commands shipped in F02–F08. Users author `.md` files with YAML frontmatter (title, description, variables) plus a Markdown body; commands are invoked with `/command-name`; variables like `{{ARG1}}`, `{{SELECTION}}`, `{{CURRENT_FILE}}`, `{{CWD}}`, `{{ENV.NAME}}`, `{{FILE:path}}` are substituted at runtime via single-pass regex; file changes are watched via fsnotify and the registry hot-reloads. Both `/commands` slash and `helixcode commands` cobra subcommand expose `list`/`show <name>`/`reload`/`run <name>`.

2. Architecture

Add `internal/commands/markdown_commands.go` (in the same package as built-ins so user-defined commands satisfy the existing `Command` interface and live in the same `Registry`). Loader scans project `.helix/commands/*.md` and user `~/.config/helixcode/commands/*.md` (project overrides user by name). Frontmatter parsed via existing `gopkg.in/yaml.v3`. fsnotify watcher (already a transitive dependency) re-runs the loader on file create/modify/delete events. New `MarkdownCommand` type implements `Command`, with `Execute()` performing variable substitution and returning the rendered text as `CommandResult.Output`. Single-pass regex `\{\{([A-Z_][A-Z0-9_]*(\.[A-Za-z_][A-Za-z0-9_]*)?(:[^}]+)?)\}\}` matches all token forms; resolver dispatches per-token-class.

3. Components

3.1 New files

- `helix_code/internal/commands/markdown_commands.go` — `MarkdownCommand`, `MarkdownLoader`, `tokenResolver`, frontmatter parser, regex substitution
- `helix_code/internal/commands/markdown_commands_test.go` — unit tests
- `helix_code/internal/commands/markdown_watcher.go` — fsnotify-driven reloader (separate file for cross-platform isolation)
- `helix_code/internal/commands/markdown_watcher_test.go`

- helix_code/internal/commands/commands_command.go — /commands slash command (list/show/reload/run)
- helix_code/internal/commands/commands_command_test.go
- helix_code/cmd/cli/commands_cmd.go — helixcode commands cobra subcommand
- helix_code/cmd/cli/commands_cmd_test.go
- helix_code/internal/commands/builtin/commands_register_test.go
- helix_code/tests/integration/markdown_commands_test.go
- challenges/p1-f09-slash-commands/CHALLENGE.md + run.sh

3.2 Modified

- helix_code/internal/commands/builtin/register.go — RegisterBuiltinCommandsWithMarkdown(registry, loader); add "commands" to GetBuiltinCommandNames + builtin_test.go skip set
- helix_code/cmd/cli/main.go — construct MarkdownLoader, start watcher, register /commands slash + helixcode commands cobra dispatcher

3.3 Types

```
// In internal/commands/markdown_commands.go
type MarkdownCommand struct {
    name        string
    title       string
    description string
    body        string          // Markdown body after frontmatter
    variables   map[string]string // declared in frontmatter
    sourcePath  string          // file path, for diagnostics
}

func (c *MarkdownCommand) Name() string         { return c.name }
func (c *MarkdownCommand) Aliases() []string    { return nil }
func (c *MarkdownCommand) Description() string  { return
        c.description }
func (c *MarkdownCommand) Usage() string        { return "/" +
        c.name + " [args]" }
func (c *MarkdownCommand) Execute(ctx context.Context, cc
        *CommandContext) (*CommandResult, error)

type MarkdownLoader struct {
    projectDir string
    userDir    string
    registry   *Registry
    mu         sync.RWMutex
    loaded     map[string]string // command name → source path
}

func NewMarkdownLoader(registry *Registry, projectDir, userDir
        string) *MarkdownLoader
```

```
func (l *MarkdownLoader) Load() error
func (l *MarkdownLoader) Reload() error
func (l *MarkdownLoader) Watch(ctx context.Context) error  //
        blocks; cancel ctx to stop
```

3.4 Frontmatter format

```
---
title: Refactor file
description: Rename a function across the file and run tests
variables:
  function_name: ""
---

Please rename function `{{ARG1}}` (or `{{ARG.function_name}}
        `) in {{CURRENT_FILE}} to `{{ARG2}}`.
After the rename, run `{{FILE:scripts/run-tests.sh}}` and report
        the diff.

Working directory: {{CWD}}
User: {{ENV.USER}}
```

3.5 Token classes (regex `\{\{([A-Z_][A-Z0-9_]*(\.[A-Za-z_][A-Za-z0-9_]*)?(:[^}]+)?)\}\}`)

- `{{ARG1}}`…`{{ARGN}}` — positional args (1-indexed)
- `{{ARG.name}}` — named arg from frontmatter `variables:` map (defaults if user didn't supply)
- `{{SELECTION}}` — current selection (CommandContext should expose `Selection string`); empty string if absent
- `{{CURRENT_FILE}}` — current file path (CommandContext.CurrentFile); empty if absent
- `{{CWD}}` — current working directory (`os.Getwd()`)
- `{{ENV.NAME}}` — env var lookup; empty string if unset (with warning logged once per Load)
- `{{FILE:path}}` — read file contents at path, with size cap (1 MB) and error on missing/oversized

4. Data flow

4.1 Startup

```
main.go
  ├─ loader := NewMarkdownLoader(cmdRegistry, ".helix/commands", "~/.confi
  ├─ loader.Load()   // scans both dirs, registers MarkdownCommand for eac
  ├─ go loader.Watch(ctx)   // fsnotify on both dirs
  └─ cmdRegistry.Register(commands.NewCommandsCommand(loader))   // /comma
```

4.2 User invokes `/refactor file.go oldName newName`

```
slash router resolves "refactor" via cmdRegistry.Get
  └─ MarkdownCommand.Execute(ctx, &CommandContext{Args: ["file.go","oldNam
       ├─ build tokenResolver from cc.Args + frontmatter variables + os.Ge
       ├─ regex.ReplaceAllStringFunc(body, resolver.Resolve)
       └─ return &CommandResult{Success: true, Output: rendered}
```

4.3 fsnotify watch

```
loader.Watch(ctx)
  └─ on Create/Modify/Delete event in either dir:
       ├─ debounce 200ms
       └─ loader.Reload()
            ├─ rescan both dirs
            ├─ diff against l.loaded (added / changed / removed)
            ├─ for added/changed: parseFrontmatter → registry.Register (
            └─ for removed: registry.Unregister(name)   // new method on
```

4.4 `helixcode commands list`

```
helixcode commands list
  └─ scans both dirs (no Registry needed; pure fs operation)
  └─ table: NAME | TITLE | SOURCE-PATH
```

`helixcode commands show <name>` reads the file and prints the body. `helixcode commands run <name> <args...>` builds a context, calls `MarkdownCommand.Execute`, prints the rendered output (so users can pipe / redirect).

5. Error handling, edge cases

- **Frontmatter parse error**: log a WARN with file path + error, skip that file (don't fail the whole Load).
- **Duplicate command name**: project file wins (deterministic precedence). User file is logged as overridden.
- `{{FILE:path}}` **over 1 MB**: substitution returns `[FILE TOO LARGE: <path>]` (visible in output, agent can retry with smaller scope).
- `{{FILE:path}}` **missing**: substitution returns `[FILE NOT FOUND: <path>]`.
- `{{ARGN}}` **out of bounds** (user supplied fewer args): substitution returns empty string (lenient — agent fills in from context).
- `{{ENV.NAME}}` **unset**: substitution returns empty string + WARN logged (once per Load).
- **fsnotify errors** (e.g., dir doesn't exist initially): logged at INFO; loader retries on next Reload trigger.
- **Watcher Goroutine leak on shutdown**: `Watch(ctx)` returns when ctx cancels.

Anti-bluff (CONST-035 / §11.9)

- Challenge: write a `.helix/commands/echo-input.md` with body `Got: {{ARG1}}`, run `helixcode commands run echo-input hello`, capture stdout `Got: hello` (not "rendered template" or "would render").
- Tests use real files via `t.TempDir()` + real fsnotify events — no mocks.

- Anti-bluff smoke must remain clean.

6. Testing

Unit tests (markdown_commands_test.go): - TestParseFrontmatter (valid + missing + malformed) - TestSubstitute_PositionalArg - TestSubstitute_NamedArg - TestSubstitute_Selection_CurrentFile_CWD - TestSubstitute_EnvVar (set + unset) - TestSubstitute_FileToken (exists + missing + oversize) - TestMarkdownCommand_ImplementsInterface (compile-time)

Loader tests: - TestLoader_LoadProjectAndUser (project overrides user) - TestLoader_LoadIgnoresNonMarkdown - TestLoader_ReloadDiffs (added/changed/removed) - TestLoader_FrontmatterErrorIsLogged (does not fail Load)

Watcher tests (markdown_watcher_test.go): - TestWatcher_DebouncesRapidWrites - TestWatcher_StopsOnContextCancel - TestWatcher_HandlesMissingDirInitially

Slash command tests (commands_command_test.go): - TestSlashCommands_List - TestSlashCommands_ShowReturnsBody - TestSlashCommands_ReloadRefreshesRegistry - TestSlashCommands_RunRendersOutput - TestSlashCommands_UnknownSubcommandErrors

Cobra tests (commands_cmd_test.go): - TestCommandsCmd_List - TestCommandsCmd_Show - TestCommandsCmd_Run

Integration test (real fs): - TestMarkdownCommands_ProjectOverridesUser (real tempdirs) - TestMarkdownCommands_WatcherReloadsOnFileWrite

Challenge: real `.helix/commands/echo.md` + `helixcode commands run echo "hello world"` → captures `hello world` in real stdout.

7. Cross-platform

fsnotify works on Linux/macOS/Windows. Cross-compile linux is the canonical check (Windows pre-existing CGO failures unrelated). User dir resolution uses `os.UserConfigDir()` (cross-platform).

8. Out of scope (deferred)

- Argument validation against frontmatter `variables:` schema (currently lenient — empty string fallback).
- Git-tracked vs untracked command provenance.
- Sandboxed execution of commands that include shell directives (commands are pure templates; no execution semantics).

9. Constitutional compliance

- CONST-035 / §11.9: Challenge captures real rendered output via real subprocess.
- CONST-042: command files may include `{{ENV.NAME}}` for sensitive vars; loader does NOT log resolved env values.
- CONST-043: non-force pushes to all four remotes per programme convention.

10. Open questions resolved during brainstorming

Q	Answer
Q1: where does loader live	(C) `internal/commands/markdown_commands.go` (same package as built-ins)
Q2: variable substitution scope	(A) Full parity (ARGN/named/SELECTION/CURRENT_FILE/ CWD/ENV/FILE)
Q3: discovery scope	(C) Project + user + fsnotify watch
Q4: user surface	(C) Both `/commands` slash + `helixcode commands` cobra
Q5: substitution implementation	(B) Single-pass regex with token resolver